You've seen how to improve your Active Record query code in a variety of ways. Now you'll explore how to directly write SQL within Active Record.

Reduced Object Allocations With SQL In Active Record

While writing Active Record query code typically involves writing Ruby, Active Record also supports SQL. This is a hybrid strategy and is a useful technique where memory or speed is very important.

Active Record has a number of methods that support direct SQL. The first one you'll look at is .find_by_sql(). With this method, instead of Ruby Active Record code, you may use SQL as text as an argument.

The results are mapped to instances of the calling class and stored in a Ruby Array. Normally Active Record returns results in a ActiveRecord::Relation which enables lazy evaluation and chaining. One trade-off with .find_by_sql() is that you won't get lazy evaluation or be able to chain more methods on.

Try running this code from bin/rails console:

```
TripRequest.find_by_sql("SELECT * FROM trips LIMIT 1")
```

Another way to run SQL in Active Record is with the .execute() method. With this method the results are mapped to "primitive types" in Ruby like the String or Integer type.

Experiment with this method by running this code from bin/rails console:

```
ActiveRecord::Base.connection.execute("SELECT * FROM trips LIMIT 1")
```

The results are placed into a PG::Result class instance. This is a bit lower level than Active Record. Take note of fields like ntuples=1 and nfields=7 which indicate one row (tuple) was returned with 7 fields of data.

When primitive types are acceptable or even desired for the response, .select_all() and .select_one() can be good options.

.select_one() works like .select_all() but returns only the first item.

With primitive types you won't incur the performance penalty of instantiating Active Record objects because that process does not happen. You'll need to determine whether the results from these methods are compatible with where you wish to use the result data.

To better understand the memory savings opportunity, use Ruby benchmarking techniques as demonstrated in *Optimizing Active Record queries*[33] to

33. https://dev.to/kattyacuevas/optimizing-active-record-queries-4i84